Navic: night-before revision

Pitch: C++17 API + React/Leaflet map + real OSM graph; Dijkstra and A* minimize road distance for driving, walking and cycling in a bounded Delhi pilot.

Pipeline: cached OSM extract → Python converter → compressed NDJSON → immutable C++ adjacency lists → validate → component-aware node snapping → shortest path → GeoJSON + ETA + steps → React/Leaflet.

Key numbers: 309,697 nodes; 708,631 directed edges; 19,907 places. Bbox S=28.50, W=76.98, N=28.70, E=77.25. Mode bits 1/2/4. Maximum snap 800 m per endpoint. Four workers. 180 requests/minute per peer default. 16 KiB body limit. 64 cached routes, each at most 64 KiB serialized. Data timestamp: 2026-11-01T19:03:08Z.

Algorithms: relaxation candidate=g(u)+w(u,v). Dijkstra priority=g. A* priority=g+h. h=great-circle distance to target. Nonnegative edge costs; edge length≥endpoint haversine; h(target)=0. Stop when the current-best target is popped, not first discovered. Parent edge IDs preserve parallel-edge choice.

Complexity: adjacency lists O(V+E). Lazy binary heap O((V+E)log(E+1)) worst-case; commonly O((V+E)log V) on sparse road graphs. Search arrays O(V), heap up to O(E). Snapping O(V). Gazetteer scan O(P), followed by sorting matches. Union-find components near-linear.

Formulas: $a=\sin^2(\Delta\varphi/2)+\cos\varphi_1\cos\varphi_2\sin^2(\Delta\lambda/2)$; $d=2R\ \operatorname{asin}(\sqrt{a})$, R=6,371,008.8 m; degrees×π/180=radians. ETA=Σ length/(speed_kmh/3.6). Consistency: $h(u)\le w(u,v)+h(v)$.

Coordinates: input objects use lat/lon. GeoJSON uses [lon,lat]. Leaflet uses [lat,lng]. WGS84 is geographic coordinates; Web Mercator is display projection. Tiles are pictures, not the routing graph.

Endpoints: GET /health, /coverage, /geocode?q=NSUT, /reverse-geocode?lat=28.61&lon=77.04, /network, /openapi.json, /docs. POST /route with source, destination, mode and algorithm.

Codes: 200 success; 204 preflight; 400 malformed JSON/query; 403 origin denied; 404 no route/unknown path; 413 oversized body; 415 content type; 422 invalid value/coverage/snap; 429 rate limit; 500 unexpected failure.

Commands: `python3 scripts/unpack_data.py`; `sh scripts/build.sh`; `./build/navic-api`; `.venv/bin/python -m pytest -q`; `cd frontend && pnpm install --frozen-lockfile && pnpm run build`; `python3 scripts/smoke_test.py "$NAVIC_API_URL"`.

Deployment: Docker runtime; bind 0.0.0.0 and platform PORT; frontend VITE_API_BASE_URL is public/build-time; ALLOWED_ORIGINS must contain exact HTTPS frontend and backend origins. Render free: 512 MB, 0.1 CPU, sleep after 15 minutes, 750 free instance hours/workspace/month (checked 2 Nov 2026). Actual public URL and cloud memory still need verification.

Never overclaim: no live traffic; no turn restrictions/barriers/conditional-access model; no full NCR coverage; no exact address geocoder; snapping gaps excluded; weak connectivity≠directed reachability; CORS≠authentication; local benchmark≠cloud SLA. Offline fallback means local C++ API + built frontend + bundled roads.

Demo: NSUT → Vegas Mall, Drive/A*: about 2.893 km / 7 min; switch to Dijkstra (same cost), Walk (about 36 min), show /docs and tests, then Offline roads. Explain data/version and limitations before claiming production reliability.